{ }
03

Von C zu C++

Verstehen. Anwenden. Nachschlagen.

Ein Lernskript zu Namensräumen, Referenzen, Klassen und sicherer Ressourcenverwaltung.

01 **SPRACHGRUNDLAGEN**
Namen, Funktionen, const

02 **OBJEKTE VERSTEHEN**
Klassen, Lebensdauer, RAII

03 **WISSEN ANWENDEN**
Operatoren und Selbsttest

Dein Lernziel
Du kannst Namen eindeutig zuordnen, Funktionsüberladungen unterscheiden und mit Zeigern, Referenzen und const umgehen. Darauf aufbauend verstehst du Klassen, Konstruktoren, Destruktoren und Operatorüberladung.

Dein Weg durch die Übung

13 Kapitel - von den Sprachgrundlagen bis zum eigenen Zahlentyp. Die Einträge führen direkt zum jeweiligen Abschnitt.

1. Namensräume und Sichtbarkeit . . . 3
2. Funktionen überladen und Default-Argumente . . . 5
3. Konstanten und Zeiger . . . 7
4. Wertübergabe, Zeiger und Referenzen . . . 9
5. Zeichenketten mit std::string . . . 11
6. Ein- und Ausgabe mit Streams . . . 12
7. Von struct zu class . . . 14
8. Die Klasse Person verstehen . . . 15
9. Konstante Memberfunktionen und Rückgabewerte . . . 18
10. Lebensdauer, Destruktoren und RAII . . . 20
11. Header und pragma once . . . 22
12. Rational - Operatoren und friend . . . 23
13. Wissen überprüfen . . . 27

Diese überarbeitete Fassung basiert auf Übung_3. Die Erklärungen zu `Person` wurden mit den vorhandenen Übungsdateien abgeglichen. Das `Rational`-Beispiel ist eine ergänzte Lernimplementierung, da die zugehörige Vorlage noch keine Implementierung enthält.

> **Codebeispiele verwenden**
>
> Die Beispiele sind für **C++17 oder neuer** geschrieben; `std::print` und `std::println` werden gesondert als C++23-Funktionen erwähnt. Blöcke mit `main()` sind eigenständige Programme. Andere Blöcke zeigen Ausschnitte und benötigen den jeweils beschriebenen Kontext. Absichtlich ungültige Zeilen sind auskommentiert.

KAPITEL 01

Namensräume und Sichtbarkeit

Namensräume ordnen Bezeichner

Ein **Namespace** gruppiert Namen, etwa von Variablen, Funktionen und Klassen. Dadurch können verschiedene Programmteile denselben kurzen Namen verwenden, ohne dass ihre vollständigen Namen gleich sind.

C++ CODEAUSSCHNITT

```cpp
namespace swc3 {
    int counter = 43;
    int x = 40;
}
```

Die vollständigen Namen lauten `swc3::counter` und `swc3::x`. `::` heißt **Scope-Resolution-Operator** und legt fest, in welchem Gültigkeitsbereich ein Name gesucht wird.

Schreibweise	Bedeutung
`x`	Unqualifizierte Namenssuche: im aktuellen und gegebenenfalls in umgebenden Gültigkeitsbereichen
`::x`	`x` im globalen Namensraum
`swc3::x`	`x` im Namensraum `swc3`
`std::cout`	`cout` aus dem Namensraum der C++-Standardbibliothek
`Person::get_name`	Member `get_name` der Klasse `Person`

Auch `std::printf` verwendet zwei Doppelpunkte; die passende Headerdatei ist `<cstdio>`.

Ein einzelner Name mit using

C++ VOLLSTÄNDIGES BEISPIEL

```cpp
#include <iostream>

int x = 10;

namespace swc3 {
    int x = 40;
}

int main() {
    using swc3::x;

    std::cout << x << '\n';        // 40
    std::cout << ::x << '\n';      // 10
    std::cout << swc3::x << '\n';  // 40
}
```

`using swc3::x;` ist eine **using-Deklaration**. Sie macht diesen einen Namen im betreffenden Gültigkeitsbereich verfügbar. Im Beispiel verdeckt er dort den globalen Namen `x`; die globale Variable bleibt unverändert vorhanden.

`using namespace swc3;` ist dagegen eine **using-Direktive**, die den Namensraum bei der Namenssuche berücksichtigt. Sie kann Namenskonflikte begünstigen. Insbesondere in Headerdateien sollte man auf globale using-Direktiven verzichten.

> **Kein textuelles Ersetzen**
>
> `using` ist kein Makro und ersetzt nicht einfach jedes `x`. Das Beispiel funktioniert durch den inneren Gültigkeitsbereich von `main()`. Direkt neben einem globalen `int x` wäre ein globales `using swc3::x;` ein Namenskonflikt.

static hängt vom Kontext ab

Außerhalb von Funktionen kann `static` einer Variablen oder Funktion **interne Bindung** geben:

C++ CODEAUSSCHNITT

```cpp
namespace swc3 {
    static int counter = 43;

    static void increment_counter() {
        ++counter;
    }
}
```

Diese Namen bezeichnen Einheiten, die nicht über externe Bindung aus anderen Übersetzungseinheiten angesprochen werden können. Eine **Übersetzungseinheit** ist vereinfacht eine `.cpp`-Datei einschließlich der durch `#include` eingefügten Inhalte. „Nur innerhalb der Datei“ ist dafür eine nützliche, aber ungenaue Kurzform. Ein weitergereichter Zeiger könnte beispielsweise trotzdem Zugriff auf das Objekt ermöglichen.

Kontext	Bedeutung von `static`
Variable/Funktion auf Namensraumebene	Interne Bindung
Lokale Variable in einer Funktion	Bleibt zwischen Aufrufen erhalten; eine nötige dynamische Initialisierung erfolgt beim ersten Erreichen der Deklaration
Datenmember einer Klasse	Gehört zur Klasse, nicht einzeln zu jedem Objekt
Memberfunktion einer Klasse	Benötigt kein Objekt und besitzt keinen `this`-Zeiger

Für interne Namen verwendet man in `.cpp`-Dateien häufig auch einen **unbenannten Namensraum**:

C++ CODEAUSSCHNITT

```cpp
namespace {
    int counter = 43;
}
```

KAPITEL 02

Funktionen überladen und Default-Argumente

Überladen: gleicher Name, unterschiedliche Parameter

Funktionen können denselben Namen haben, wenn sich ihre Parameterlisten geeignet unterscheiden, beispielsweise in Anzahl oder Typ der Parameter. Der Compiler wählt anhand des Aufrufs die am besten passende Funktion.

C++ VOLLSTÄNDIGES BEISPIEL

```cpp
#include <iostream>

void print(int value) {
    std::cout << "Zahl: " << value << '\n';
}

void print(const char* text) {
    if (text == nullptr) {
        std::cout << "Kein Text vorhanden\n";
        return;
    }
    std::cout << "Text: " << text << '\n';
}

int main() {
    print(42);       // print(int)
    print("Hallo");  // print(const char*)
    print(0);        // print(int): 0 hat den Typ int
    print(nullptr);  // print(const char*)
}
```

Das Integerliteral `0` kann zwar auch als Nullzeigerkonstante dienen, aber bei `print(0)` passt `int` exakt. Diese Übereinstimmung ist besser als die Umwandlung in einen Zeiger.

`nullptr` ist seit C++11 die eigene Nullzeigerkonstante. Sie lässt sich in Zeigertypen umwandeln, aber nicht in `int`. Deshalb wählt sie hier die Zeigerüberladung. Bei mehreren passenden Zeigerüberladungen kann auch `nullptr` zu Mehrdeutigkeit führen.

> **Nullzeiger bewusst behandeln**
>
> Verwende `nullptr`, wenn du „zeigt auf kein Objekt“ ausdrücken willst. Die aufgerufene Funktion muss diesen Fall unterstützen; `nullptr` darf nicht als gültige C-Zeichenkette gelesen werden.

Der **Rückgabetyp allein** genügt nicht zur Überladung:

C++ CODEAUSSCHNITT

```cpp
int calculate(int x);
// double calculate(int x);  // Fehler: gleiche Parameterliste
```

Default-Argumente ersetzen fehlende Argumente

C++ CODEAUSSCHNITT

```cpp
void print(int a = 5);
void print(int a, int b);

void demo() {
    print();      // erste Funktion, a = 5
    print(3);     // erste Funktion, a = 3
    print(3, 4);  // zweite Funktion
}
```

`print(3)` **ist hier eindeutig.** Die zweite Funktion benötigt zwei Argumente und hat keinen Default-Wert für `b`. Sie kommt deshalb für diesen Aufruf nicht infrage. Diese Auswahlregel ist im C++-Entwurf zu aufrufbaren Kandidaten beschrieben.

Mehrdeutig wäre dagegen dieses **separate** Beispiel:

C++ CODEAUSSCHNITT

```cpp
void print(int a = 5);
void print(int a, int b = 0);

void demo() {
    // print(3);  // Fehler: beide Funktionen passen gleich gut
}
```

Für gewöhnliche Funktionsparameter gilt: Wenn ein Parameter einen Default-Wert hat, müssen auch die rechts folgenden Parameter Default-Werte besitzen. Außerdem sollte man einen Default-Wert an einer zentralen, für Aufrufer sichtbaren Deklaration angeben, meist im Header. In der Definition wird er nicht wiederholt.

C++ CODEAUSSCHNITT

```cpp
// Deklaration, typischerweise im Header:
void show(int value = 5);

// Definition, typischerweise in der .cpp-Datei:
void show(int value) {
    // ...
}
```

KAPITEL 03

Konstanten und Zeiger

const schützt ein Objekt vor Änderungen

Diese beiden Schreibweisen sind gleichbedeutend:

C++ CODEAUSSCHNITT

```cpp
const int a = 3;
int const b = 3;

// a = 4;  // Fehler: a ist const
// b = 4;  // Fehler: b ist const
```

const_cast macht ein konstantes Objekt nicht veränderbar

C++ CODEAUSSCHNITT

```cpp
const int value = 3;
int* ptr = const_cast<int*>(&value);

// *ptr = 4;  // Undefiniertes Verhalten: value ist tatsächlich const!
```

Der Cast selbst entfernt hier die `const`-Qualifikation aus dem Zeigertyp. Ein anschließender Schreibzugriff auf das tatsächlich konstante Objekt hat jedoch **undefiniertes Verhalten**. Es gibt keine Zusicherung, dass danach „4 in `value` steht“; das Programm hat dann kein vom Sprachstandard festgelegtes Verhalten. Siehe C++-Entwurf zu cv-Qualifikatoren.

Ein anderer Fall ist ein ursprünglich **veränderbares** Objekt, das nur über einen Zeiger auf `const` betrachtet wird:

C++ CODEAUSSCHNITT

```cpp
int value = 3;
const int* read_only = &value;

int* writable = const_cast<int*>(read_only);
*writable = 4;  // Erlaubt: das zugrunde liegende Objekt ist nicht const
```

In diesem einfachen Fall könnte man direkt `value = 4;` schreiben. Ein `const_cast` ist kein üblicher Ersatz für eine passende Schnittstelle.

Konstanter Zeiger oder Zeiger auf const?

C++ CODEAUSSCHNITT

```cpp
int value = 3;
int other = 8;

int* p = &value;
int* const fixed_pointer = &value;
const int* read_only = &value;
const int* const fixed_read_only = &value;
```

Typ	Zeiger auf anderes Objekt setzen?	Wert über diesen Zeiger ändern?
`int*`	Ja	Ja
`int* const`	Nein	Ja
`const int*` oder `int const*`	Ja	Nein
`const int* const`	Nein	Nein

C++ CODEAUSSCHNITT

```cpp
*fixed_pointer = 4;    // Erlaubt
// fixed_pointer = &other;  // Fehler

read_only = &other;    // Erlaubt
// *read_only = 5;     // Fehler
```

> **const beschreibt auch den Zugriffsweg**
>
> Ein `const int*` bedeutet nicht zwingend, dass das Zielobjekt selbst konstant ist. Es bedeutet, dass man es **über diesen Zeiger** nicht ändern darf. Ein Zeiger benötigt außerdem eine gültige Adresse oder `nullptr`; eine Zahl wie `3` ist kein gültiger gewöhnlicher Zeigerinitialisierer.

Wert erhöhen oder Zeiger weiterschieben?

Ausdruck	Wirkung
`(*p)++` oder `++(*p)`	Erhöht den Wert des Objekts, auf das `p` zeigt
`p++` oder `++p`	Verschiebt den Zeiger um ein Element
`*p++` beziehungsweise `*(p++)`	Verwendet zum Dereferenzieren den bisherigen Zeigerwert und schiebt `p` weiter
`*++p` beziehungsweise `*(++p)`	Schiebt `p` weiter und dereferenziert den neuen Zeigerwert

Bei Präfix- und Postfix-`++` unterscheidet sich außerdem der Wert des Ausdrucks: Präfix liefert den erhöhten, Postfix den bisherigen Wert.

C++ VOLLSTÄNDIGES BEISPIEL

```cpp
#include <iostream>

int main() {
    int values[] = {10, 20, 30};
    int* p = values;

    ++(*p);          // values[0] wird 11
    int old = *p++;  // old wird 11; p zeigt danach auf values[1]

    std::cout << old << ' ' << *p << '\n';  // 11 20
}
```

`*(p++);` als alleinstehende Anweisung nutzt den dereferenzierten Ausdruck nicht weiter; insbesondere erhöht es **nicht** das Zielelement. Zeigerarithmetik ist nur innerhalb des zugehörigen Arrays und bis genau eine Position dahinter erlaubt. Diese Position direkt hinter dem Array darf man bilden, aber nicht dereferenzieren.

KAPITEL 04

Wertübergabe, Zeiger und Referenzen

C++ unterstützt sowohl **Wertparameter** als auch **Referenzparameter**. Die Aussage „immer Call by Value“ trifft auf C zu, aber nicht allgemein auf C++.

C++ VOLLSTÄNDIGES BEISPIEL

```cpp
#include <iostream>

void inc_value(int value) {
    ++value;  // verändert nur die lokale Kopie
}

void inc_pointer(int* value) {
    if (value != nullptr) {
        ++(*value);  // verändert das Objekt an der übergebenen Adresse
    }
}

void inc_reference(int& value) {
    ++value;  // value ist ein Alias für das Argument
}

int main() {
    int number = 10;

    inc_value(number);
    std::cout << number << '\n';  // 10

    inc_pointer(&number);
    std::cout << number << '\n';  // 11

    inc_reference(number);
    std::cout << number << '\n';  // 12
}
```

Parameter	Was erhält die Funktion?	Änderung des ursprünglichen `int` möglich?	Aufruf
`int value`	Eine Kopie des Werts	Nein	`f(number)`
`int* value`	Eine Kopie der Adresse	Ja, über `*value`	`f(&number)`
`int& value`	Eine Referenz auf das Original	Ja	`f(number)`
`const int& value`	Eine Referenz mit schreibgeschütztem Zugriff	Nicht über diese Referenz	`f(number)`

Bei einem Zeigerparameter wird **der Zeiger selbst per Wert** übergeben. `value = nullptr;` würde daher nur den lokalen Zeiger ändern. `*value = 42;` ändert dagegen das Objekt, auf das er zeigt, sofern der Zeiger gültig ist.

Eine Referenz ist ein Alias

C++ CODEAUSSCHNITT

```cpp
int a = 3;
int b = 7;
int& ref = a;

ref = b;  // weist a den Wert 7 zu; ref bleibt an a gebunden
```

Eine Referenz muss bei ihrer Initialisierung gebunden werden und kann danach nicht auf ein anderes Objekt „umgebogen“ werden. Sie besitzt keinen gültigen Nullzustand. Sie kann allerdings **ungültig werden**, wenn das referenzierte Objekt nicht mehr lebt.

> **Parameter passend wählen**
>
> Kleine Werte wie `int` übergibt man meist per Wert. Soll das Original geändert werden, eignet sich häufig `T&`. Größere Objekte, die nur gelesen werden, kann man mit `const T&` ohne Objektkopie übergeben. Ein Zeiger eignet sich unter anderem dann, wenn `nullptr` einen sinnvollen optionalen Zustand ausdrückt.

KAPITEL 05

Zeichenketten mit std::string

`std::string` aus `<string>` verwaltet eine veränderbare Zeichenkette und deren Speicher automatisch.

C++ VOLLSTÄNDIGES BEISPIEL

```cpp
#include <iostream>
#include <string>

int main() {
    std::string s1{"Klemens"};
    auto s2 = std::string{"Danner"};  // auto ermittelt std::string

    std::string s3 = s1;  // Kopierkonstruktion: neues Objekt s3
    s1 = s2;              // Kopierzuweisung: s1 existiert bereits

    s1.clear();           // nur s1 wird leer
    s1.push_back('C');    // s1 == "C"
    s1 += "++";           // s1 == "C++"
    s1.insert(0, "Ich lerne ");

    std::cout << s1 << '\n';  // Ich lerne C++
    std::cout << s2 << '\n';  // Danner
    std::cout << s3 << '\n';  // Klemens
}
```

Kopierkonstruktion und Kopierzuweisung unterscheiden

`std::string s3 = s1;` erzeugt ein neues Stringobjekt. `s1 = s2;` verändert dagegen das bereits existierende Objekt `s1`. Danach besitzen beide Strings unabhängig veränderbare Inhalte. Ob intern neuer Speicher reserviert werden muss, ist eine andere Frage.

Operation	Zweck
`s.size()`	Anzahl der gespeicherten `char`-Elemente
`s.empty()`	Prüft, ob der String leer ist
`s.clear()`	Entfernt alle Zeichen
`s.push_back('x')`	Hängt ein einzelnes Zeichen an
`s += "Text"`	Hängt eine Zeichenkette an
`s.insert(position, "Text")`	Fügt Text an einer Position ein
`s.at(index)`	Zugriff mit Bereichsprüfung; ungültiger Index löst eine Ausnahme aus

Die Positionen beginnen bei `0`. Für `insert` darf die Position auch genau `s.size()` sein; dann wird am Ende eingefügt. Bei UTF-8 entspricht die Anzahl der `char`-Elemente nicht zwingend der Anzahl sichtbarer Zeichen.

Ein **Stringliteral** wie `"Hallo"` darf nicht verändert werden. Ein daraus erzeugter `std::string` ist dagegen veränderbar, sofern das Stringobjekt nicht selbst als `const` deklariert wurde.

KAPITEL 06

Ein- und Ausgabe mit Streams

Die Headerdatei `<iostream>` stellt unter anderem diese Streams bereit:

Stream	Verwendung
`std::cout`	Standardausgabe
`std::cin`	Standardeingabe
`std::cerr`	Fehlerausgabe

`<<` fügt Werte in einen Ausgabestream ein; `>>` liest Werte aus einem Eingabestream. Diese Operatoren sind für viele Datentypen überladen.

C++ VOLLSTÄNDIGES BEISPIEL

```cpp
#include <iostream>
#include <string>

int main() {
    std::string name;
    int number = 0;

    std::cout << "Dein Name: ";
    if (!std::getline(std::cin, name)) {
        std::cerr << "Name konnte nicht gelesen werden.\n";
        return 1;
    }

    std::cout << "Eine ganze Zahl: ";
    if (!(std::cin >> number)) {
        std::cerr << "Keine gueltige ganze Zahl eingegeben.\n";
        return 1;
    }

    std::cout << "Hallo, " << name
              << ". Deine Zahl ist " << number << ".\n";
}
```

Ein Wort oder eine ganze Zeile?

- `std::cin >> name` liest ein Wort bis zum nächsten Leerraum.
- `std::getline(std::cin, name)` liest eine Zeile einschließlich enthaltener Leerzeichen. Der abschließende Zeilenumbruch wird verbraucht und nicht gespeichert.

- Nach `std::cin >> number` kann der Zeilenumbruch noch im Eingabestream stehen. Ein unmittelbar folgendes `getline` würde dann oft eine leere Zeile lesen.

Nach einer **erfolgreichen** Zahleneingabe kann man den Rest dieser Eingabezeile verwerfen, bevor man zur nächsten Zeile wechselt:

C++ CODEAUSSCHNITT

```cpp
// Zusätzlich erforderlich: #include <limits>
std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
std::getline(std::cin, name);
```

Zeilenumbruch und Puffer

`'\n'` schreibt einen Zeilenumbruch. `std::endl` schreibt ebenfalls einen Zeilenumbruch und fordert zusätzlich das Leeren des Ausgabepuffers an (**Flush**). Für gewöhnliche Zeilenumbrüche genügt meist `'\n'`.

> **Namen als std::string lesen**
>
> Gegenüber `char name[10]` vermeidet ein `std::string` die selbst festgelegte kleine Puffergrenze. Das macht die Eingabe einfacher. Die verfügbaren Sicherheitsmechanismen bei Eingabe in ein Zeichenarray unterscheiden sich außerdem je nach C++-Version und konkretem Typ des Eingabeziels.

Im vorhandenen `Person.cpp` wird auch `std::print` beziehungsweise `std::println` aus `<print>` verwendet. Diese formatierte Ausgabe wurde mit **C++23** eingeführt; Compiler und Standardbibliothek müssen sie unterstützen. Zum Beispiel formatiert `std::println("Hallo, {}", name);` den Namen und ergänzt einen Zeilenumbruch. Siehe den WG21-Vorschlag zu formatierter Ausgabe.

KAPITEL 07

Von struct zu class

Ein struct kann Memberfunktionen besitzen

In C++ kann ein `struct` sowohl Daten als auch Funktionen enthalten:

C++ VOLLSTÄNDIGES BEISPIEL

```cpp
#include <iostream>

struct Point {
    int x;
    int y;

    void draw() const;  // Deklaration einer Memberfunktion
};                      // Semikolon nach der Typdefinition!

void Point::draw() const {
    std::cout << '(' << x << ", " << y << ")\n";
}

int main() {
    Point origin{};     // x = 0, y = 0
    Point p{1, 2};      // x = 1, y = 2

    origin.draw();      // (0, 0)
    p.draw();           // (1, 2)
}
```

`Point::draw` ordnet die außerhalb definierte Funktion der Klasse `Point` zu. Beim Aufruf wird das betreffende Objekt mitgegeben; innerhalb der Funktion kann man auf dessen Member zugreifen.

> **Initialisierung beachten**
>
> Für genau dieses `Point` initialisiert `Point p{};` beide Zahlen mit `0`. Ein lokales `Point p;` ohne Initialisierer lässt die beiden `int`-Member dagegen uninitialisiert. Sie dürfen dann vor einer Wertzuweisung nicht gelesen werden.

In C dürfen Strukturen keine Memberfunktionen besitzen. Ein C-Struct kann aber einen **Funktionszeiger als Datenmember** enthalten. Ein solcher Member belegt Speicher im jeweiligen Objekt. Eine gewöhnliche nichtvirtuelle C++-Memberfunktion wird dagegen nicht als Funktionszeiger in jedem Objekt gespeichert. Virtuelle Funktionen können zusätzlichen Verwaltungsaufwand verursachen; die genaue Umsetzung ist compilerabhängig.

struct und class unterscheiden sich in den Vorgaben

Eigenschaft	`struct`	`class`
Standardzugriff auf Member	`public`	`private`
Standardzugriff bei Vererbung	`public`	`private`
Konstruktoren, Destruktoren und Memberfunktionen	Möglich	Möglich
Explizites `public`, `protected`, `private`	Möglich	Möglich

Beide definieren in C++ Klassentypen. Häufig verwendet man `struct` für einfache Datenzusammenfassungen und `class` für Typen mit gekapseltem Zustand und eigenen Regeln. Das ist eine Konvention, keine Sprachvorschrift.

`private` **gilt auf Klassenebene, nicht pro Objekt:** Eine Memberfunktion darf auch auf private Member eines anderen Objekts derselben Klasse zugreifen. Das wird später bei `Rational::operator+=` wichtig.

KAPITEL 08

Die Klasse Person verstehen

Die vorhandene Person.h deklariert die Schnittstelle; Person.cpp enthält die Implementierung. Das folgende eigenständige Beispiel führt beides zusammen und verwendet für die Ausgabe `std::cout`:

C++ VOLLSTÄNDIGES BEISPIEL

```cpp
#include <iostream>
#include <string>

namespace swc3 {

class Person {
public:
    explicit Person(const std::string& name = "");
    virtual ~Person();

    void set_name(const std::string& name);
    [[nodiscard]] const std::string& get_name() const;
    void print() const;

private:
    std::string m_name;
};

Person::Person(const std::string& name) : m_name(name) {
    std::cout << "Erzeugt: " << m_name << '\n';
}

```

C++ VOLLSTÄNDIGES BEISPIEL / FORTSETZUNG

```cpp
Person::~Person() {
    std::cout << "Zerstoert: " << m_name << '\n';
}

void Person::set_name(const std::string& name) {
    m_name = name;
}

const std::string& Person::get_name() const {
    return m_name;
}

void Person::print() const {
    std::cout << "Person(" << m_name << ")\n";
}

}  // namespace swc3

int main() {
    swc3::Person p1{"Klemens"};
    swc3::Person p2;  // name erhält den Default-Wert ""

    p2.set_name("Alex");
    p1.print();
    std::cout << p2.get_name() << '\n';
}  // zuerst p2, dann p1 zerstören
```

Konstruktor und Initialisierungsliste

Ein **Konstruktor** stellt den Anfangszustand eines Objekts her. Er hat denselben Namen wie die Klasse und keinen Rückgabetyp.

In `Person::Person(const std::string& name) : m_name(name)` bedeutet:

1. `Person::Person`: Definition des Konstruktors der Klasse `Person`.
2. `const std::string& name`: Der Parameter greift ohne Kopie des übergebenen Stringobjekts auf den Namen zu. Wird ein Stringliteral übergeben, entsteht dafür zunächst ein temporärer `std::string`.
3. `: m_name(name)`: **Member-Initialisierungsliste**; `m_name` wird direkt aus `name` kopierkonstruiert.
4. Erst anschließend wird der Funktionsrumpf ausgeführt.

Ein `m_name = name;` im Rumpf wäre eine spätere Zuweisung an ein bereits initialisiertes Member. Member werden in der Reihenfolge ihrer **Deklaration in der Klasse** initialisiert, unabhängig von der Reihenfolge in der Initialisierungsliste.

`m_` ist lediglich eine Namenskonvention für Datenmember. `this->m_name` und `m_name` bezeichnen in diesen Funktionen dasselbe Member.

Default-Konstruktor und explicit

Ein **Default-Konstruktor** ist ein Konstruktor, der ohne Argumente aufgerufen werden kann. Er muss daher nicht unbedingt eine leere Parameterliste besitzen: Bei `Person` reicht der Default-Wert `name = ""` aus.

C++ CODEAUSSCHNITT

```cpp
// Im Kontext der oben definierten Klasse:
swc3::Person p1{"p1"};
swc3::Person p2;
swc3::Person p3{};
```

Deklariert man einen Konstruktor mit einem zwingend erforderlichen Argument selbst, erzeugt der Compiler nicht zusätzlich automatisch einen parameterlosen Konstruktor. `Person p2;` wäre ohne den Default-Wert beziehungsweise einen weiteren passenden Konstruktor deshalb nicht möglich.

`explicit` verhindert, dass dieser Konstruktor für implizite Umwandlungen in `Person` verwendet wird:

C++ CODEAUSSCHNITT

```cpp
std::string name{"Klemens"};

swc3::Person p{name};          // Erlaubt: direkte Initialisierung
// swc3::Person q = name;     // Fehler wegen explicit
```

Getter, Setter und Kapselung

`m_name` ist privat. Code außerhalb der Klasse kann es daher nicht direkt über `p.m_name` lesen oder verändern. Dafür stehen öffentliche Funktionen bereit:

- `set_name(...)` weist dem vorhandenen String einen neuen Inhalt zu.
- `get_name()` gibt lesenden Zugriff auf den Namen.
- `print()` stellt die Person als Text dar.

`[[nodiscard]]` fordert den Compiler auf, auf einen ignorierten Rückgabewert hinzuweisen. Ein Aufruf wie `p.get_name();` ohne weitere Verwendung kann dadurch eine Warnung auslösen. Das Attribut ist keine Garantie für eine Fehlermeldung und verhindert den Aufruf nicht.

KAPITEL 09

Konstante Memberfunktionen und Rückgabewerte

Das const hinter der Parameterliste

C++ CODEAUSSCHNITT

```cpp
// Deklaration innerhalb der Klasse Person:
[[nodiscard]] const std::string& get_name() const;
```

Hier erfüllen die beiden `const` unterschiedliche Aufgaben:

Bestandteil	Bedeutung
`const std::string&`	Referenz auf den Namen; über sie darf der String nicht geändert werden
`get_name() const`	Die Memberfunktion darf die gewöhnlichen Datenmember des aktuellen Objekts nicht über `this` ändern

Eine solche Memberfunktion kann sowohl auf konstanten als auch auf nichtkonstanten Objekten aufgerufen werden:

C++ CODEAUSSCHNITT

```cpp
const swc3::Person person{"Klemens"};
std::cout << person.get_name() << '\n';  // Erlaubt
// person.set_name("Alex");              // Fehler: Setter ist nicht const
```

Das abschließende `const` gehört zur Funktionsdeklaration und muss auch bei einer getrennten Definition erscheinen:

C++ CODEAUSSCHNITT

```cpp
class Counter {
public:
    int get_value() const {
        // m_value = 10;  // Fehler: Änderung über const-this
        return m_value;
    }

private:
    int m_value = 0;
};
```

> **const ist keine allgemeine Nebenwirkungsfreiheit**
>
> Eine `const`-Memberfunktion darf beispielsweise etwas ausgeben. Mit `mutable` markierte Member können verändert werden; auch ein Zeiger-Member macht das Objekt, auf das er zeigt, nicht automatisch konstant. Entscheidend ist der Schutz der gewöhnlichen Member über den Zugriffsweg `this`.

Rückgabe als Wert oder Referenz

Rückgabetyp	Ergebnis	Worauf achten?
`T`	Eigenständiger Ergebniswert	Je nach Ausdruck Konstruktion, Kopie, Verschiebung oder vermiedene Kopie
`T&`	Veränderbare Referenz auf ein bestehendes Objekt	Das referenzierte Objekt muss weiterleben
`const T&`	Lesende Referenz auf ein bestehendes Objekt	Ebenfalls an dessen Lebensdauer gebunden

Eine Wertrückgabe ruft **nicht grundsätzlich den Kopierkonstruktor** auf. C++ kann Kopien vermeiden oder einen Wert verschieben. Bei einer Referenzrückgabe wird dagegen kein neues Objekt des referenzierten Typs erzeugt. Siehe die Regeln zur return-Anweisung.

Auch die Variable auf Aufruferseite ist entscheidend:

C++ CODEAUSSCHNITT

```cpp
swc3::Person person{"Klemens"};

auto name_copy = person.get_name();        // eigener std::string: Kopie
const auto& name_ref = person.get_name();  // Referenz auf person.m_name

person.set_name("Alex");
// name_copy bleibt "Klemens"; name_ref bezieht sich nun auf den Inhalt "Alex".
```

> **Keine Referenz auf ein zerstörtes Objekt**
>
> Eine Referenz auf `person.m_name` darf nach der Zerstörung von `person` nicht mehr verwendet werden. Ebenso darf eine Funktion keine nutzbare Referenz auf eine eigene gewöhnliche lokale Variable zurückgeben: Diese Variable lebt nach dem Funktionsende nicht mehr.

KAPITEL 10

Lebensdauer, Destruktoren und RAII

Automatische Objekte werden beim Verlassen des Blocks zerstört

C++ CODEAUSSCHNITT

```cpp
// Im Kontext der Klasse Person:
void demonstrate_lifetime() {
    swc3::Person outer{"A"};

    {
        swc3::Person inner{"B"};
    }  // Destruktor von inner

}  // Destruktor von outer
```

Das gilt bei normalem Blockende, bei `return` und beim Verlassen durch eine Ausnahme im Rahmen der Stack-Abwicklung. Bei mehreren lokalen Objekten erfolgt die Zerstörung in umgekehrter Reihenfolge der abgeschlossenen Konstruktion. Sonderfälle wie ein sofortiger Prozessabbruch führen nicht diese normale Aufräumsequenz aus. Siehe C++-Entwurf zu Destruktoren.

„Liegt auf dem Stack“ ist die übliche technische Vorstellung. Präziser spricht man hier von **automatischer Speicherdauer**. Eine lokale `static`-Variable ist davon zu unterscheiden: Sie wird nicht am Ende jedes Funktionsaufrufs zerstört.

Ein Destruktor heißt `~Klassenname()`, besitzt keinen Rückgabetyp und wird normalerweise automatisch aufgerufen. Nach seinem Rumpf werden auch die entsprechenden Member und Basisklassen zerstört. Der String `m_name` räumt seinen eigenen Speicher dabei selbst auf.

Dynamischer Speicher braucht einen Besitzer

C++ CODEAUSSCHNITT

```cpp
// Im Kontext der Klasse Person:
void manual_lifetime() {
    auto* person = new swc3::Person{"Dynamisch"};
    person->print();

    delete person;  // zerstört das Objekt und gibt seinen Speicher frei
    person = nullptr;
}
```

Die lokale Zeigervariable und das dynamisch erzeugte Objekt haben unterschiedliche Lebensdauern. Verschwindet nur die Zeigervariable, bleibt das mit `new` erzeugte Objekt bestehen. Wenn niemand mehr für seine Freigabe sorgt, entsteht ein **Speicherleck**. Ein Objekt im dynamischen Speicher ist also nicht automatisch ein Speicherleck.

RAII übernimmt das Aufräumen

RAII (*Resource Acquisition Is Initialization*) bindet den Besitz einer Ressource an ein Objekt. Dessen Destruktor gibt die Ressource frei. `std::string` verwaltet beispielsweise seinen Zeichenspeicher selbst; `std::unique_ptr` besitzt ein dynamisches Objekt:

C++ CODEAUSSCHNITT

```cpp
#include <memory>
// Zusätzlich wird die oben definierte Klasse Person benötigt.

void automatic_cleanup() {
    auto person = std::make_unique<swc3::Person>("Dynamisch");
    person->print();
}  // unique_ptr zerstört die Person und gibt deren Speicher frei
```

Der `unique_ptr` ist hier ein automatisches Objekt, die `Person` ein dynamisches Objekt. Dieses Zusammenspiel sorgt auch bei Ausnahmen für das Aufräumen. Falls keine dynamische Lebensdauer benötigt wird, genügt meist direkt ein lokales `Person`-Objekt.

Rule of Zero

Verwende nach Möglichkeit Member, die ihre Ressourcen selbst verwalten, etwa `std::string` oder passende Standardcontainer. Dann benötigt die eigene Klasse meist keine selbst geschriebenen Kopier-, Verschiebe- oder Aufräumoperationen. Bei `Person` dient der gezeigte Destruktorrumpf vor allem der sichtbaren Protokollierung.

Warum ist der Destruktor von Person virtual?

Virtuelle Funktionen ermöglichen bei Aufrufen über Basisklassenzeiger oder -referenzen die Auswahl der passenden überschriebenen Funktion anhand des tatsächlichen Objekttyps. Ein virtueller Destruktor ist insbesondere dann erforderlich, wenn ein abgeleitetes Objekt auf gewöhnliche Weise **über einen Basisklassenzeiger gelöscht** werden soll:

C++ VOLLSTÄNDIGES BEISPIEL

```cpp
#include <iostream>

struct Base {
    virtual ~Base() {
        std::cout << "~Base\n";
    }
};

struct Derived : Base {
    ~Derived() override {
        std::cout << "~Derived\n";
    }
};

int main() {
    Base* p = new Derived;
    delete p;  // zuerst ~Derived, danach ~Base
}
```

Ohne den virtuellen Destruktor in `Base` hätte dieses `delete p;` undefiniertes Verhalten. Dasselbe Grundproblem betrifft einen `std::unique_ptr<Base>`, der mit seinem üblichen Deleter ein `Derived` löscht. Siehe die C++-Regeln für delete.

> **virtual wirkt auf die betreffende Funktion**
>
> `virtual ~Person()` ermöglicht die polymorphe Zerstörung. Es macht andere Funktionen wie `print()` nicht automatisch virtuell. Ein virtueller Destruktor ist auch nicht für jede Klasse erforderlich; entscheidend ist ihre vorgesehene Verwendung als Basisklasse.

KAPITEL 11

Header und pragma once

Header werden oft indirekt mehrfach eingebunden: Eine `.cpp`-Datei inkludiert beispielsweise zwei Header, die beide `Person.h` benötigen. Ohne Schutz könnte dieselbe Klassendefinition dadurch mehrfach in einer Übersetzungseinheit erscheinen.

C++ CODEAUSSCHNITT

```cpp
// Datei: Example.h
#pragma once

class Example {
    // ...
};
```

`#pragma once` weist den Compiler an, diese Headerdatei innerhalb einer Übersetzungseinheit nur einmal einzubinden. Es wird von vielen verbreiteten Compilern unterstützt, gehört aber nicht zum C++-Standard. Die portable Alternative sind **Include Guards**, wie auch die GCC-Dokumentation zum Mehrfacheinbindungsschutz erläutert:

C++ CODEAUSSCHNITT

```cpp
// Alternative für dieselbe Datei: Example.h
#ifndef SWC3_EXAMPLE_H
#define SWC3_EXAMPLE_H

class Example {
    // ...
};

#endif  // SWC3_EXAMPLE_H
```

> **Einmal pro Übersetzungseinheit**
>
> Dieser Schutz bedeutet nicht „einmal im gesamten Programm“. Mehrere `.cpp`-Dateien können denselben Header jeweils einbinden. Gewöhnliche nicht-inline Funktionsdefinitionen gehören daher typischerweise in eine `.cpp`-Datei; ein Include Guard allein verhindert keine mehrfachen Definitionen beim Linken.

KAPITEL 12

Rational - Operatoren und friend

Eine rationale Zahl lässt sich als Bruch aus Zähler und Nenner darstellen:

$$r = \frac{n}{d}, \quad d \neq 0$$

Die Bedingung „Nenner ungleich null“ ist eine **Klasseninvariante**: Sie soll für jedes erfolgreich konstruierte Objekt gelten und durch dessen Operationen erhalten bleiben.

> **Umfang des Lernbeispiels**
>
> Die vorhandene `Rational`-Vorlage enthält noch keine Implementierung. Das folgende Beispiel ergänzt deshalb eine mögliche Lösung zum Verständnis von Klassen und Operatoren. Es kürzt Brüche nicht und vereinheitlicht Vorzeichen nicht. Es setzt kleine Zahlen voraus, deren **sämtliche Zwischen- und Endergebnisse in `int` passen**. Überlauf vorzeichenbehafteter Ganzzahlen wäre undefiniertes Verhalten; eine belastbare Bruchbibliothek müsste das zusätzlich behandeln.

Ein vollständiges Beispiel

Für die Addition verwenden wir:

$$\frac{a}{b} + \frac{c}{d} = \frac{a \cdot d + c \cdot b}{b \cdot d}$$

C++ VOLLSTÄNDIGES BEISPIEL

```cpp
#include <iostream>
#include <stdexcept>

class Rational {
public:
    explicit Rational(int numerator = 0, int denominator = 1)
        : m_numerator(numerator), m_denominator(denominator) {
        if (denominator == 0) {
            throw std::invalid_argument("Nenner darf nicht 0 sein");
        }
    }

    [[nodiscard]] int numerator() const {
        return m_numerator;
    }

    [[nodiscard]] int denominator() const {
        return m_denominator;
    }

```

C++ VOLLSTÄNDIGES BEISPIEL / FORTSETZUNG

```cpp
    Rational& operator+=(const Rational& rhs) {
        const int new_numerator =
            m_numerator * rhs.m_denominator
            + rhs.m_numerator * m_denominator;
        const int new_denominator = m_denominator * rhs.m_denominator;

        m_numerator = new_numerator;
        m_denominator = new_denominator;
        return *this;
    }

    friend std::ostream& operator<<(std::ostream& out, const Rational& value) {
        return out << value.m_numerator << '/' << value.m_denominator;
    }

private:
    int m_numerator;
    int m_denominator;
};

Rational operator+(Rational lhs, const Rational& rhs) {
    lhs += rhs;
    return lhs;
}

int main() {
    Rational a{1, 2};
    const Rational b{1, 3};

    std::cout << (a + b) << '\n';  // 5/6
    std::cout << a << '\n';        // 1/2: a wurde nicht verändert

    a += b;
    std::cout << a << '\n';        // 5/6
}
```

Die Aufgaben der einzelnen Teile

Bestandteil	Aufgabe
Konstruktor	Initialisiert Zähler und Nenner; weist Nenner `0` mit einer Ausnahme zurück
`numerator()` / `denominator()`	Lesen die privaten Werte, ohne das Objekt zu ändern
`operator+=`	Verändert den linken Operanden und gibt ihn als Referenz zurück
`operator+`	Berechnet ein neues Ergebnis; die ursprünglichen Operanden bleiben unverändert
`operator<<`	Gibt den Bruch aus und gibt den Stream für weitere Ausgaben zurück

`lhs` steht für *left-hand side* (linker Operand), `rhs` für *right-hand side* (rechter Operand).

this und return *this

In einer gewöhnlichen nichtstatischen Memberfunktion zeigt `this` auf das aktuelle Objekt. Bei `a += b` ist dieses Objekt `a`:

- `this` ist hier ein Zeiger vom Typ `Rational*`.
- `*this` bezeichnet das aktuelle `Rational`-Objekt.
- `return *this;` mit Rückgabetyp `Rational&` gibt eine Referenz auf dieses Objekt zurück.

C++ CODEAUSSCHNITT

```cpp
// Im Kontext der Klasse Rational:
Rational a{1, 2};
Rational b{1, 3};
Rational& result = (a += b);
// &result == &a: result und a bezeichnen dasselbe Objekt.
```

Die Rückgabe ist hier sicher, weil `a` außerhalb der Memberfunktion weiterlebt. Dieses Muster entspricht dem Verhalten eingebauter Zuweisungsoperatoren und erlaubt Ausdrücke wie `(a += b) += c`.

`rhs.m_numerator` ist trotz `private` zugänglich: `operator+=` ist eine Memberfunktion von `Rational` und darf private Member **aller** `Rational`-Objekte ansprechen. Die Berechnung beider neuen Werte vor den Zuweisungen macht außerdem auch `a += a` korrekt, solange die Zahlen im genannten Wertebereich bleiben.

operator+ baut auf operator+= auf

Bei `a + b` wird `a` in den Wertparameter `lhs` kopiert. `lhs += rhs` verändert diese lokale Kopie; `a` und `b` bleiben unverändert. Anschließend wird das Ergebnis **als Wert** zurückgegeben. Ein `Rational&` wäre hier falsch: Der lokale Parameter `lhs` lebt nach dem Aufruf nicht weiter.

Drei Varianten einer binären Operatorüberladung

Ein binärer Operator wie `+` kann auf unterschiedliche Weise umgesetzt werden:

Variante	Bedeutung von `a + b`	Zugriff auf private Member
Memberfunktion	`a.operator+(b)`	Durch Zugehörigkeit zur Klasse
Freie Funktion	`operator+(a, b)`	Über die öffentliche Schnittstelle
Freie `friend`-Funktion	`operator+(a, b)`	Zusätzlich ausdrücklich erlaubt

Die Deklarationen sähen alternativ so aus:

C++ CODEAUSSCHNITT

```cpp
// Variante 1: innerhalb der Klasse
// Rational operator+(const Rational& rhs) const;

// Variante 2: außerhalb der Klasse
// Rational operator+(Rational lhs, const Rational& rhs);

// Variante 3: friend-Deklaration innerhalb der Klasse
// friend Rational operator+(Rational lhs, const Rational& rhs);
```

Diese Varianten sind **alternative Entwürfe** für dieselbe Operation. Man sollte nicht gleichzeitig gleich gut passende Member- und freie Überladungen anbieten, weil Aufrufe dann mehrdeutig werden können. Im vollständigen Beispiel ist `+` eine freie Funktion ohne `friend`: Sie benötigt nur das öffentliche `operator+=`.

Bei freien binären Operatoren sind beide Operanden gewöhnliche Parameter; zulässige implizite Umwandlungen können daher auf beiden Seiten berücksichtigt werden. Ein Memberoperator benötigt links dagegen ein Objekt, für das dieser Member gefunden wird. Im gezeigten Beispiel unterbindet `explicit` die implizite Umwandlung von `int` zu `Rational`; für einen solchen Operanden schreibt man ausdrücklich etwa `Rational{2}`.

friend gewährt gezielten Zugriff

`friend` kann einer bestimmten Funktion oder einer Klasse Zugriff auf private und geschützte Member erlauben. Eine freie Funktion wird dadurch **nicht** zu einer Memberfunktion und erhält keinen `this`-Zeiger. Siehe C++-Entwurf zu friend.

Der Ausgabeoperator ist deshalb eine passende freie Funktion: Bei `std::cout << a` steht links ein `std::ostream`, kein `Rational`. Als `friend` kann er rechts auf `value.m_numerator` und `value.m_denominator` zugreifen. Alternativ könnte er ohne Freundschaft die öffentlichen Getter benutzen.

Die Rückgabe `std::ostream&` ermöglicht Verkettung:

C++ CODEAUSSCHNITT

```cpp
// Im Kontext des Rational-Beispiels:
std::cout << a << " + " << b << '\n';
```

Jeder Teilausdruck gibt denselben Stream zurück, an den der nächste `<<`-Aufruf anknüpft.

> **Bedeutung der Operatoren erhalten**
>
> `+` sollte ein neues Ergebnis liefern, `+=` das linke Objekt verändern. Operatorüberladung definiert das Verhalten für eigene Typen; sie ändert weder die Vorrangregeln noch die Anzahl der Operanden. Verwende Operatoren so, dass ihr Verhalten zur gewohnten Bedeutung passt.

KAPITEL 13

Wissen überprüfen

Beantworte die Fragen zunächst ohne nachzusehen. Die Lösungen findest du gesammelt auf der folgenden Seite.

1. Warum ruft `print(0)` bei den Überladungen `print(int)` und `print(const char*)` die Integerfunktion auf?
2. Ist `print(3)` bei `print(int a = 5)` und `print(int a, int b)` mehrdeutig?
3. Was unterscheidet `const int*` von `int* const`?
4. Warum darf man ein tatsächlich konstantes Objekt auch nach `const_cast` nicht verändern?
5. Was verändert `++(*p)` und was verändert `p++`?
6. Warum ändert `inc_reference(number)` das Original, `inc_value(number)` aber nicht?
7. Erzeugt `s1 = s2` ein neues `std::string`-Objekt namens `s1`?
8. Warum ist `Person p;` mit dem gezeigten Konstruktor möglich?
9. Was bedeuten die zwei `const` in `const std::string& get_name() const`?
10. Wann braucht eine Basisklasse einen virtuellen Destruktor?
11. Warum kann `Rational::operator+=` auf die privaten Member von `rhs` zugreifen?
12. Warum gibt `operator+=` eine Referenz, `operator+` aber einen Wert zurück?

EIGENE ANTWORTEN UND NOTIZEN

Lösungen und Merksätze

Lösungen

1. Das Literal `0` hat den Typ `int`; diese exakte Übereinstimmung ist besser als die Umwandlung in einen Zeiger.
2. Nein. Der zweiten Funktion fehlt das erforderliche Argument für `b`.
3. Bei `const int*` ist der Zugriff auf den Zielwert schreibgeschützt; bei `int* const` ist die gespeicherte Adresse unveränderlich.
4. Der Cast ändert nur den Zugriffsweg. Ein Schreibzugriff auf das tatsächlich konstante Objekt hat undefiniertes Verhalten.
5. `++(*p)` erhöht den Zielwert. `p++` schiebt den Zeiger um ein Element weiter, sofern die Arraygrenzen das zulassen.
6. Der Referenzparameter ist an das Original gebunden; der Wertparameter ist eine Kopie.
7. Nein. Es weist dem bestehenden Objekt `s1` den Inhalt von `s2` zu.
8. Sein Konstruktor ist durch das Default-Argument `name = ""` ohne Argumente aufrufbar.
9. Das vordere schützt den String über die zurückgegebene Referenz; das hintere beschränkt Änderungen am aktuellen Objekt über `this`.
10. Insbesondere wenn abgeleitete Objekte mittels gewöhnlichem `delete` über einen Zeiger auf diese Basisklasse zerstört werden sollen.
11. Der private Zugriff wird auf Klassenebene geregelt; die Memberfunktion darf auch auf andere Objekte derselben Klasse zugreifen.
12. `+=` verändert ein bereits bestehendes Objekt, das weiterlebt. `+` erzeugt ein eigenständiges Ergebnis; eine Referenz auf seine lokale Kopie würde nach dem Aufruf ungültig.

Merksätze

- **Sichtbarkeit:** `::` ordnet Namen zu; `using` steuert die Namenssuche.
- **Übergabe:** Wertparameter kopieren, Referenzparameter binden an ein Objekt.
- **Konstanz:** Unterscheide konstantes Objekt, konstanten Zugriffsweg und konstanten Zeiger.
- **Klassen:** Konstruktoren initialisieren; Destruktoren räumen beim Lebensdauerende auf.
- **Ressourcen:** Besitz mit RAII ausdrücken, damit Aufräumen automatisch geschieht.
- **Operatoren:** Gewohnte Bedeutung und passende Rückgabetypen beibehalten.